07/10/2026

CSA1407 - Complier Design

CO1 - Assessment Tool 1

Name : Rashmitha.

Reg NO : 192525034

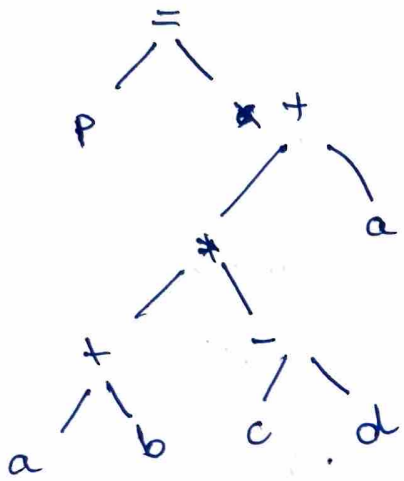
a) Identifiy Tokens :

$P = (a+b) * (c-d) + a$

S.NO	Token	Name
1.	(	Symbol
2.	a	Identifier
3.	+	Operator
4.	b	Identifier
5.	)	Symbol
6.	*	operator
7.	(	Symbol
8.	c	Variable (Identifier)
9.	-	Operator
10.	d	variable
11.	)	Symbol
12.	+	Operator
13.	a	Identifier
14.	P	Identifier
15.	=	Operator.

Here there are 15 Tokens in the given Expression.

b) Syntax Tree:



c) Intermediate code:

$t_1 = a + b$
$t_2 = c - d$
$t_3 = t_1 * t_2$
$t_4 = t_3 + a$
$t_5 = p$

d) Optimization code:

$t_1 = a + b$
$t_2 = c - d$
$t_3 = t_1 * t_2$
$p = t_3 + a$

The unnecessary temporary t_4 is removed.

e) Target code:

A Simple assembly like expression:

```
MOV  R1, a
ADD  R1, b
MOV  R2, C
SUB  R2, D
MUL  R1, R2
ADD  R1, a
MOV  P, R$
```

2. i) $0(0+1)^*$

Valid String:

{0, 00, 01, 001, 0101}

Invalid strings: {ε, 1, 10, 110, 101}

ii) $(a+b)^*$ abb:

valid string: {abb, aabb, babb, ababb, bbabb}

Invalid string: {ε, ab, aba, bba, aab}

iii) 1^*01^*

Valid String: {0, 10, 01, 110, 011}

Invalid String: {ε, 1, 00, 1010, 001}

iv) $a^*(ba^*)^*$

valid string: {ε, a, b, ab, ba}

There is no invalid string because the expression generates every possible string.

v) $(0+1)^*00(0+1)^*$

valid Strings: {00, 000, 1001, 0011, 11000}

Invalid Strings: {ε, 0, 1, 01, 1010}

```
main()
{
    int x = 25;
    float y = 12.5;
    char ch = 'A';
    int total = x+10;
    float z = 20.5;
    char name = "ABC";
    x = 12abc;
}
```

a) Token:

S.NO	Tokens	Name
1.	main	Keyword
2.	(	Symbol
3.	)	Symbol
4.	&	Symbol
5.	int	Keyword
6.	x	variable
7.	=	operator
8.	25	constant
9.	;	delimiter
10.	float	Keyword
11.	y	Identifier
12.	=	operator
13.	12.5	constant
14.	;	delimiter
15.	char	Keyword
16.	ch	Identifier
17.	=	operator
18.	'A'	string
19.	;	delimiter
20.	int	Keyword
21.	total	Keyword
22.	=	operator
23.	x	Identifier
24.	+	operator
25.	10	Constant
26.	;	delimiter
27.	flor	Keyword

28.	at	Keyword
29.	z	Identifier
30.	=	operator
31.	20.5	constant
32.	;	delimiter
33.	char	Keyword
34	name	Identifier
35.	=	
36.		operator
37.	"ABC"	string
38.	;	Delimiter
39.	x	Identifier
40	=	operator
41.	12	constant
42.	a	Identifier
43.	b	Identifier
44.	c	Identifier
45.	;	Delimiter
	}	Symbol.

∴ There are 45 TOKens in th

ii) Lexical Errors:

⇒ Error 1: float

flo at z = 20.5;

The intended keyword is float z = 20.5;

Because of the space, flo and at are treated as separate Identifiers instead of keyword float.

⇒ Corrected form: float z = 20.5;

⇒ Error 2: 12abc.

Corrected Example could be:

x = 12; or x = abc;

depending on the intended meaning.

⇒ And Char name = "ABC"; uses a string literal. So, if the intention is a string character it should be:

Char name = 'A';

ii) classifications of Tokens:

* Keyword:
  int, float, char

* Identifier:
  main, x, y, ch, total, flo;at, z, name

* Constants:
  25, 12.5, 'A', 10, 20.5, "ABC", 12abc → invali

* Operators:
  =, +

* Separators:
  (,), {, }, ;

Lexically correct statements:

Corrected program statements:

```
int x =25;
float y = 12.5;
char ch = 'A';
int total = x +10;
float z = 20.5;
char name = "ABC";
x = 12;
```

Regular Expression for the Given Languages:

i) Even number of 1s

$\Sigma = \{0, 1\}$

Regular Expression: $0^*(10^*10^*)^*$

Language $L = \{\varepsilon, 0, 00, 11, 101, 01010 \ldots\}$

ii) At least two 0's

Regular Expression:

$$(0+1)^*0(0+1)^*0(0+1)^*$$

$L = \{1\,0001, 00100, 00001 \ldots\}$

iii) Beginning with 0 and ending with 1.

$L = \{01, 001, 011, 0101, 00101\}$

R.E $= 0(0+1)^*1$

iv) No occurrence of 00

$L = \{\varepsilon, 0, 1, 01, 101, 0101, \ldots\}$

R.E $= (1+01)^*(\varepsilon+0)$

Odd number of 0's and even number of 1's

$L = \{0, 011, 101, 11001, 01100 \ldots\}$

Regular Expression :

$P = (00+11)$

$Q = (01+10)$

Given :

```
int count25 = 100;
float average = 25.50;
if (count25>=50){
        count25 = count25+10;
        return;
}
```

a) Total number of Token :

1. int - Keyword
2. count25 - Identifier
3. = - operator
4. 100 - Constant
5. ; - delimiter
6. float - Keyword
7. average - Keyword
8. 25.50 - Constant

9. ; - delimiter

10. if - Keyword

11. (- symbol

12. count25 - Identifier

13. >= - operator

14. 50 - constant

15.) - symbol

16. { - symbol

17. count25 - Identifier

18. = - operator

19. count25 - Identifier

20. + - operator

21. 10 - constant

22. ; - delimiter

23. return - keyword

24. ; - delimiter

25. } - symbol

26.

∴ There are 25 Tokens

---

b) Keywords :- int, float, if, return.

operators :- =, >=, +

separator :- ;, (,), {, }

Identifiers :- counts, average

Integer Literals :- 100, 50, 10

Float - point Literals :- 25.50

c) FSM :

The lexical analyzer uses FSM to recognize different tokens :

* Identifier : start → Letter → Letter | Digit → Accept
* Integer : Start → Digit → Digit → Accept
* Float : Start → Digit → Digit → Accept
* Keyword : Recognized like an identifier, then checked with the Keyword table.
* Operators : Recognized by reading one or more operator symbols.

---

d) State Transitions :

* Identifier → 3 basic states
* Integer → 2 . 3 states
* Float → 5 - 6 states
* = → 1 state
* >= → 2 to 3 states
* Separators → 1 state each

The exact number depend on the FSM design.

e) Look - Ahead :

Tokens require look-ahead are :

* >= - to distinguish it from >.
* 25.50 - to distinguish a float-point literal from an integer.